Technical Report

Blockchain-Based Library Book Lending Tracker

Final Summative Project: Design and Implementation of a Blockchain-Based Library Book Lending Tracker for ALU

Introduction to Blockchain Development (Software Engineering, Low-Level Specialization)

Student: Joseph Nishimwe

Date: 13 October 2026

GitHub: Summative_Blockchain_LibraryBookLendingTracker

Demo video: https://www.youtube.com/watch?v=ma2JMULPHJY

Project files

file	Purpose of the file
config.h	All the library's rules in one place: loan period, fine per day, renewal limit, block size, difficulty range, rewards and mining fees.
crypto.c , crypto.h	Security with OpenSSL: SHA-256, ECDSA (secp256k1) key generation, signing and verification, encrypted key files, PIN hashing.
keystore.c , keystore.h	One key pair per member and librarian in data/keys. Private keys are encrypted and loaded only for the moment they sign.
registry.c , registry.h	Loads books, members and librarians from the data files, checks ID formats, and finds a record by its ID.
inventory.c , inventory.h	Physical copies, the availability state machine, and the reservation queue of each title.
merkle.c , merkle.h	Builds the Merkle tree of a block from scratch.
blockchain.c , blockchain.h	Lending records, blocks, block hashing, proof of work, and full chain validation.
pending.c , pending.h	The pending pool and its priority ordering.
ledger.c , ledger.h	Tokens in both models: UTXO (inputs, outputs, change) and account-based (balances, nonces, history linked list).
lending.c	Builds, checks and signs every request, and applies records once they are mined.
fraud.c	The four misuse heuristics.
mining.c , mining.h	Builds a block from the pool and confirms it by solo, pool or cloud mining. Also the difficulty benchmark.
storage.c , storage.h	Saves and loads the whole state in one file, with a signed seal.
library.c , library.h	Start-up, the validation gate, and saving.
cli.c , cli.h , main.c	The command shell, the login and the command-line options. The commands themselves are grouped in four files that share cli_internal.h: cli_catalog.c (members, books, copies), cli_lending.c (lending, reservations, fines), cli_tokens.c (transfers, balances, history) and cli_chain.c (pool, mining, chain, fraud review, audit).
books.txt, copies.txt, members.txt, librarians.txt books.txt , copies.txt , members.txt , librarians.txt	The registries: books, physical copies, members, and staff with a PBKDF2 hash of their PIN.

Makefile	Builds the whole program and links OpenSSL with one command (make).

1. Introduction

This report describes a library lending system built for ALU. The system keeps track of who borrowed which book, when it is due back, and what happened to it afterwards. What makes it different from an ordinary library system is that every action is written to a blockchain, which means that a record cannot be quietly changed once it has been made.

The reason for this design is trust. In a normal library, a librarian could mark a lost book as returned, a borrower could dispute what the paper log says, and a due date could be altered after the fact. Nobody would be able to prove what really happened. In this system, every borrow, return, reservation, renewal, overdue flag, fine payment and token transfer is stored as a signed record, which means it carries proof of who made it. New records first wait in a queue called the pending pool. They become permanent when they are confirmed in a mined block, and after that point any attempt to change them is detected by the program.

The report is written for both technical and non-technical readers. Each topic is first explained in everyday language and then shown as it is implemented in the program, with screenshots taken from the running system. Technical names such as file names, commands and field names are kept exactly as they appear in the code, so that they can be checked against the source.

2. Blockchain implementation in this project

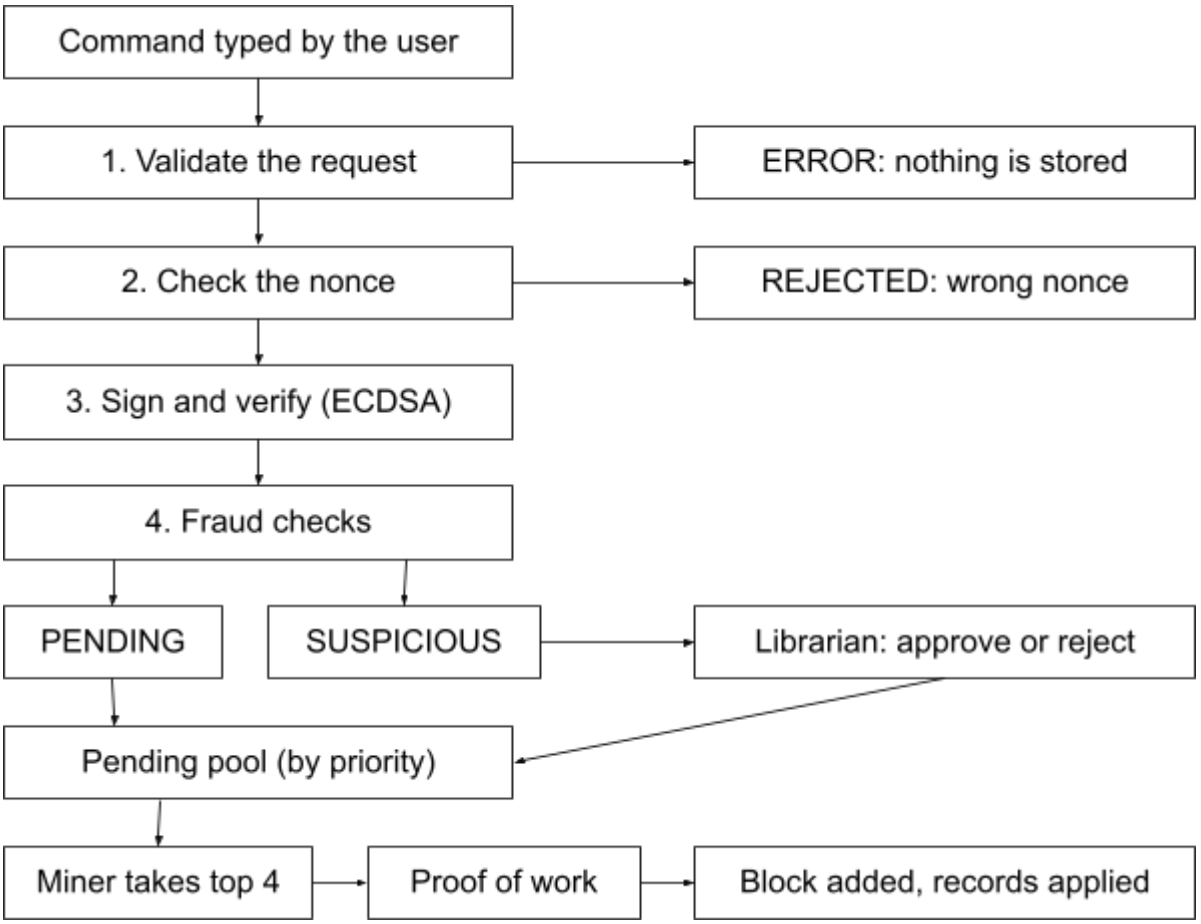
A blockchain is a list of records grouped into blocks. Each block carries a digital fingerprint of the block before it, so the blocks form a chain. If anyone edits an old block, its fingerprint changes and no longer matches the one stored in the next block, so the change is noticed straight away. The tables below explain the few terms that are needed to follow the rest of this report, first in plain English and then as they appear in this project.

term	**What it does**	**Role in the project**
Record	One signed library action	A borrow, return, reservation, renewal, overdue flag, fine payment or transfer
Block	A sealed group of records	Up to 4 records

term	**What it does**	**Role in project**
Hash	A 64-character fingerprint of some data. Change one letter and it changes completely	SHA-256
Merkle root	One fingerprint that covers all the records of a block	merkle_root field
Chain	Each block stores the hash of the block before it	previous_hash field

Signature	Proof of who created a record, which cannot be forged without a secret key	ECDSA on the secp256k1 curve
Nonce	A counter. In mining it is the number being tried; for a member it is the serial number of their next request	Two different fields
Mining	Doing a deliberate amount of computer work before a block is accepted	Proof of Work, difficulty 1 to 4
Token	A coin that exists only as an entry in the records	Reward for returning a book

Every request, whether it is a borrow, a return or a token transfer, goes through the same six steps, and the sections of this report follow them in order. First, the command that the user types builds a record and checks it against the registries, the state of the copy and the lending rules. Second, the program checks that the record carries the signer’s next nonce, which is a serial number that stops an old request from being submitted twice. Third, the record is signed with the signer’s private key and the signature is verified. Fourth, the fraud checks run: a request that looks doubtful enters the pool marked SUSPICIOUS, and every other request enters it marked PENDING. Fifth, a miner takes the four PENDING requests with the highest priority, builds a block from them and does the proof of work. Sixth, and only at this point, the records take effect: copies change state, reservation queues move, rewards are paid, nonces advance and the whole state is saved to disk. The diagram below shows the same path.

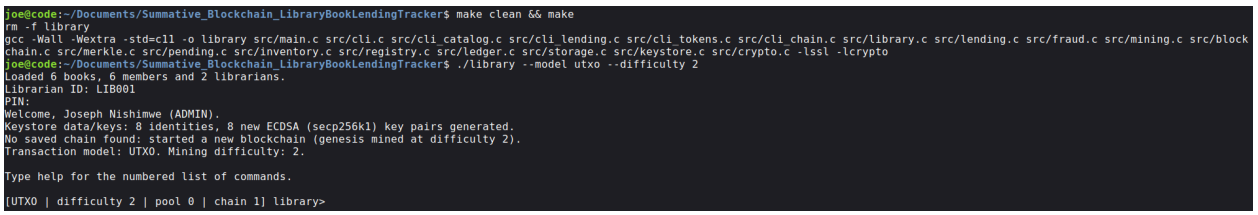


The life of a request: validation, nonce check, signing, fraud screening, the prioritised pool, mining and confirmation.

3. Project structure and building

The code is split into 19 modules, and each module has one job. This keeps the library rules separate from the blockchain logic and from the cryptography, so that one part can be read or changed without touching the others. All the rules that can be tuned, such as the loan period or the fine per day, are kept in one file, config.h. Changing a value there and rebuilding the program changes the behaviour everywhere.

The program needs gcc, make and OpenSSL 3. One command builds it and one starts it:



The build and the first start-up. The code compiles with -Wall and -Wextra and prints no warnings. On the first run the program creates one ECDSA key pair per identity and mines a genesis block.

The prompt is worth watching through the rest of this report. It always shows the active token model, the mining difficulty, how many requests are in the pool, and how long the chain is.

4. Blockchain architecture and data structures

The block

A block is a sealed group of lending records together with a header that describes it. In the code it is a C struct called Block, defined in blockchain.h. It holds the fields required by the project brief, plus the two fields used by proof of work. The table below lists each field and what it holds.

Index	What it holds
index	Position in the chain. 0 is the genesis block
timestamp	Unix time when the block was created. Always later than the previous block's
record_count	Number of lending records in the block (0 to 4)
previous_hash	SHA-256 hash of the previous block. This is the link of the chain
merkle_root	Computed from the SHA-256 hashes of all records in the block
hash	SHA-256 of all header fields and the mining nonce
sealed_by	The miner that sealed the block: a librarian ID, POOL-M3, CLOUD-RIG
token_reward	Coins credited for this block's returns: 10 per on-time return, 5 per late return
transaction_id	SHA-256 over the block's reward transaction IDs
difficulty, nonce	The number of leading zeros required, and the nonce that satisfied it
records[4]	The lending records themselves

The block hash covers every header field. The records are covered through the Merkle root, so changing any record changes the root, which changes the hash, which breaks the link to the next block.

The lending record

A lending record describes one library action, such as one borrow or one return. In the code it is a struct called LendingRecord. It holds the fields that the brief requires: record_id, book_id, copy_id, book_title, member_id, member_name, action, due_date, fine_amount, nonce and signature. The book title and the member name are copied from the registries at the moment the request is made. The record also keeps a few extra details that the program needs: the time of the request, the ID and public key of the person who signed it, the condition of a copy when it comes back, a reason code when a librarian is involved, and the reward fields.

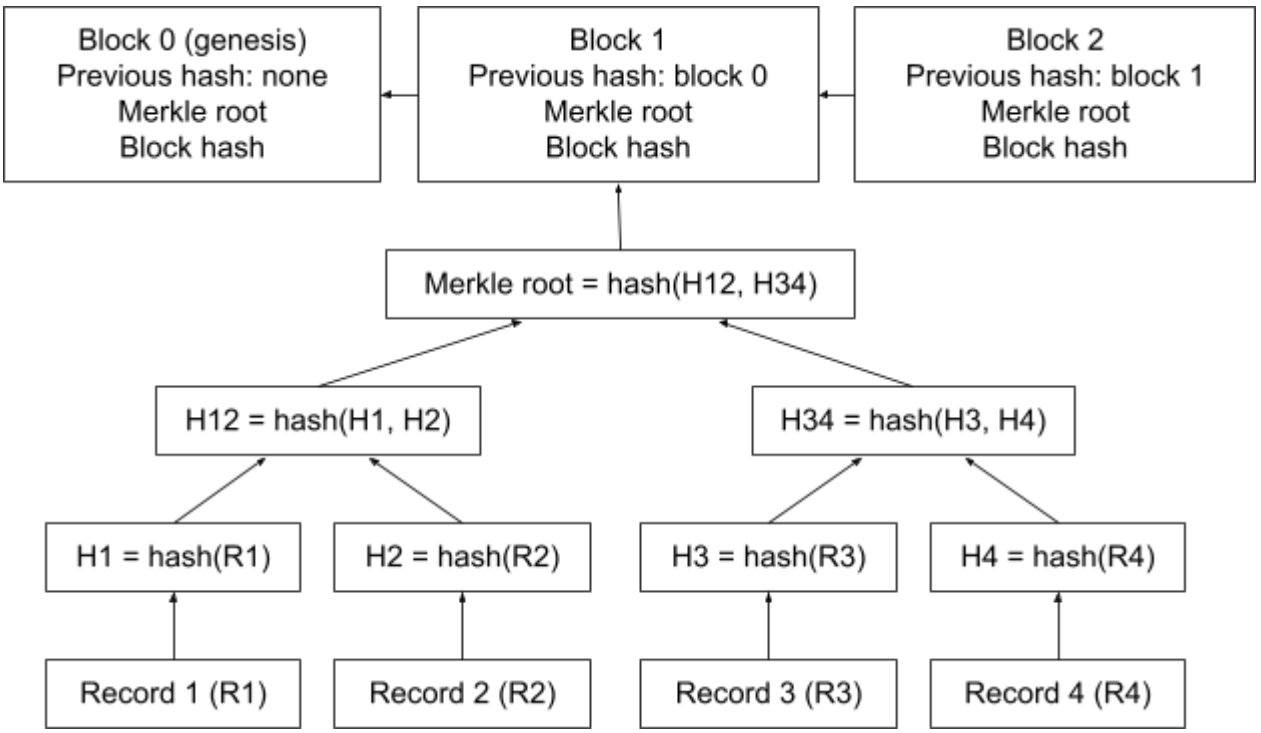
The brief lists five actions: BORROWED, RETURNED, RESERVED, RENEWED and OVERDUE. I added three more, so that these also pass through the pool and mining: **LOST** (the librarian override), **SETTLED** (a fine payment) and **TRANSFER** (a token transfer).

The chain and the pool

The program keeps two lists in memory. The first is the chain itself, which is the list of blocks that have already been mined. The second is the pending pool, which is the list of requests that are still waiting. Both are stored as arrays on the heap, which simply means that the program asks for more memory as the lists grow, so their length is not fixed in advance. Each request in the pool holds its record, with the request_id, book_id, copy_id, member_id and the action requested. It also holds a priority, which decides how soon it is mined, and a status, which is PENDING, CONFIRMED or SUSPICIOUS.

Several records per block: fixed-size batching

A block does not hold just one record. Records are grouped, and this project groups them by a fixed size: a miner takes up to 4 records from the top of the pool for each block. The number 4 is set by BLOCK_CAPACITY in config.h. If the pool holds more than 4 requests that are ready to be mined, mining simply continues with a second block. The other common choice is a time window, where a block is closed after a set amount of time, whatever it contains. I chose the fixed size because it is predictable: the same pool always produces the same blocks, which makes the system easy to test and to demonstrate.



Blockchain structure. Each block points to the previous block's hash, and the Merkle root in the header summarises the block's records.

```
[UTXO | difficulty 2 | pool 4 | chain 1] library> mine_solo

=== SOLO MINING by LIB001 (difficulty 2: hash must start with 2 zeros) ===

Block #1 candidate: 4 requests taken from the top of the pool: REQ-000001 REQ-000002 REQ-000003 REQ-000004
Mined block #1 after 219 hash attempts (nonce 218, 1.1 ms)

------------------------------------------------------------------------
Block #1  [LENDING]  4 records, sealed by LIB001
  Time        : 2026-10-07 16:29
  PoW         : difficulty 2, nonce 218
  Hash        : 009995fef6d2d493c597bc170920c5981a43555341caa3179caeca7908781103
  Prev        : 00f0c8bcc36be6e67f27a64348de8d82981b58c96bceebf794e22a7e175bcda1
  Merkle root : 106703b8f7ca6213c774222e9d49851b1bd55f0d339e1c34720a58713fcd4bbd
  Reward      : 0 coins for returns
    REQ-000001 BORROWED BK001-C1  'The Money Trap: Lost Illusions Inside th'  ALU001 (Irakoze Jean)
               at 2026-10-07 16:29 | due 2026-10-21 16:29
               nonce 0 | signed by ALU001 (key 04e3a0d852e0a7f9...) | signature VALID
    REQ-000002 BORROWED BK003-C1  'Sustainable Leadership'  ALU002 (Joseph Habimana)
               at 2026-10-07 16:29 | due 2026-10-21 16:29
               nonce 0 | signed by ALU002 (key 0442b9656c8eaf49...) | signature VALID
    REQ-000003 BORROWED BK004-C1  'Ancient Philosophy: The Fundamentals'  ALU005 (Mukamana Alice)
               at 2026-10-07 16:29 | due 2026-10-21 16:29
               nonce 0 | signed by ALU005 (key 04fdbf9ceb3a4026...) | signature VALID
    REQ-000004 BORROWED BK002-C1  'Wars Guns & Votes: Democracy in Dangerou'  ALU004 (Manzi Eric)
               at 2026-10-07 16:29 | due 2026-10-21 16:29
               nonce 0 | signed by ALU004 (key 04731c5bf6ab5b38...) | signature VALID
Confirmed 4 records:
  -> copy BK001-C1 is BORROWED by ALU001, due 2026-10-21 16:29.
  -> copy BK003-C1 is BORROWED by ALU002, due 2026-10-21 16:29.
  -> copy BK004-C1 is BORROWED by ALU005, due 2026-10-21 16:29.
  -> copy BK002-C1 is BORROWED by ALU004, due 2026-10-21 16:29.
4 confirmed requests removed from the pending pool (0 left).

UTXO set (0 unspent outputs):
TX ID                Out  Owner            Amount
-------------------- ----  ------------ ----------
Total supply                                 0.00

--- Solo mining summary ---
Blocks confirmed      : 1
Total hash attempts   : 219 (average 219 per block)
Mining reward to LIB001: 50.00 coins (1 x 50), on top of the member token rewards

[UTXO | difficulty 2 | pool 0 | chain 2] library>
```

A real block with four records. The hash starts with 00 (difficulty 2), Prev is the hash of the genesis block, and every record shows its nonce, its signer and a VALID signature.

5. Merkle tree construction and verification

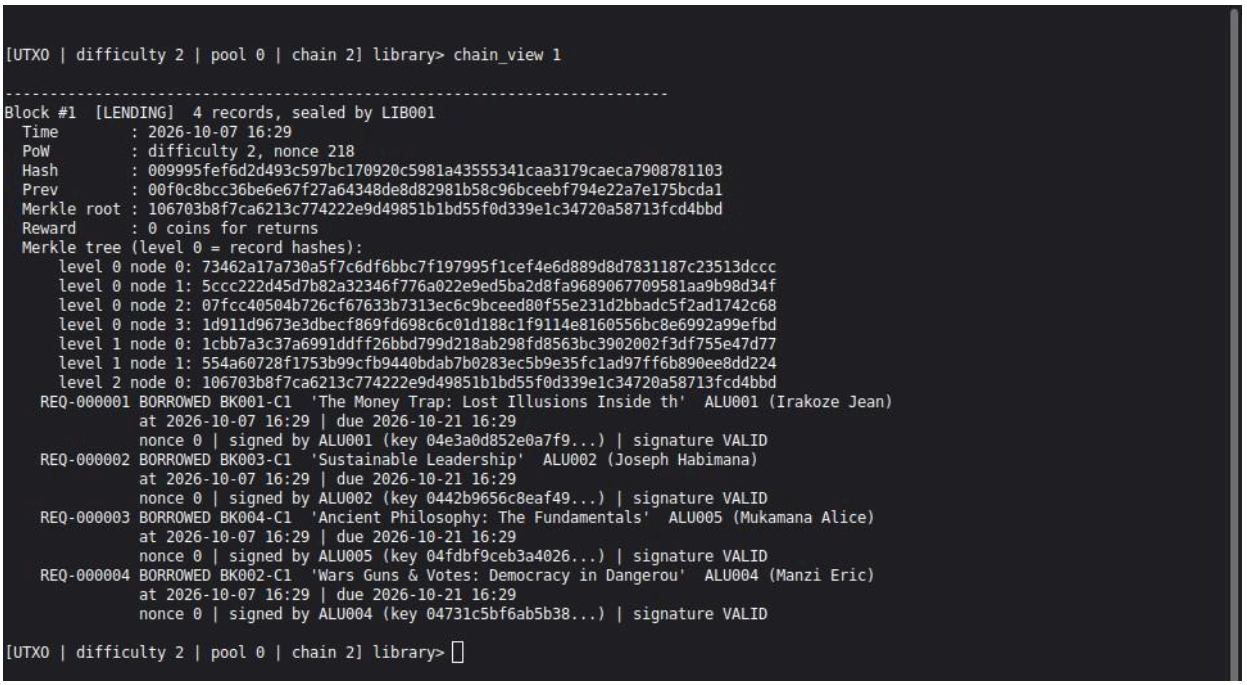
A Merkle tree is a way of summarising many records with one fingerprint. If any record in the block changes, even by a single character, the fingerprint changes too. This lets the program check a whole block by comparing one value. The tree is built from scratch in merkle.c, with no external Merkle library.

Construction.

The tree is built from the bottom up. Each record is first turned into a leaf, which is the SHA-256 hash of its signed fields together with its signature. The leaves are then hashed in pairs: two 32-byte hashes are joined and hashed again to produce their parent. If a level has an odd number of nodes, the last node is paired with itself. This is repeated level by level until a single hash is left, and that hash is the Merkle root. The code keeps one working array and reduces it in place, so a block of four records needs only one small allocation.

How it is used.

The Merkle root is used in three places. It is written into the block header as merkle_root before mining starts, so the proof of work covers it. It is recomputed from the records and compared with the header every time a miner builds a block and every time a block is loaded from disk. Finally, validate_chain recomputes the root of every block to detect tampering: if one character of one record was changed, the leaf changes, so the root changes, and the program reports “Merkle root does not match the records”. The command chain_view followed by a block number prints every level of that block’s tree, as the screenshot below shows.



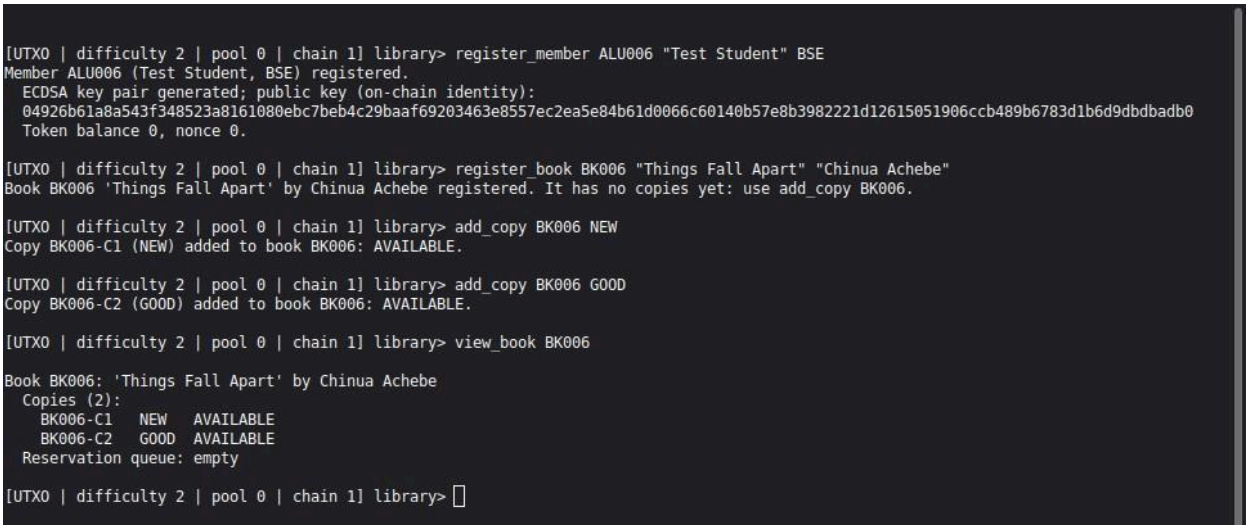
The Merkle tree of block #1. Four leaves (level 0) become two nodes (level 1) and one root (level 2). The root equals the Merkle root field of the header.

6. Copy inventory model and the availability state machine

A library often owns several copies of the same book. Formative 1 tracked a whole title as either “on loan” or “free”, which cannot describe a title where one copy is out and another is on the shelf. In this version each physical copy is tracked on its own, so a title with several copies can be partly available.

Copies are loaded from copies.txt, where each line gives a copy_id, a book_id and a condition, and new copies can be added with the add_copy command. For each copy the program stores its

copy_id and the book_id of the title it belongs to. It stores the condition of the copy, which is NEW, GOOD, WORN or LOST, and its state, which is AVAILABLE, BORROWED, RESERVED or LOST. It also stores the current_holder_id, which is the ID of the member who has the copy and is empty when nobody has it. Finally it stores the due_date, the number of renewals so far, and whether an OVERDUE record has already been raised for the current loan.



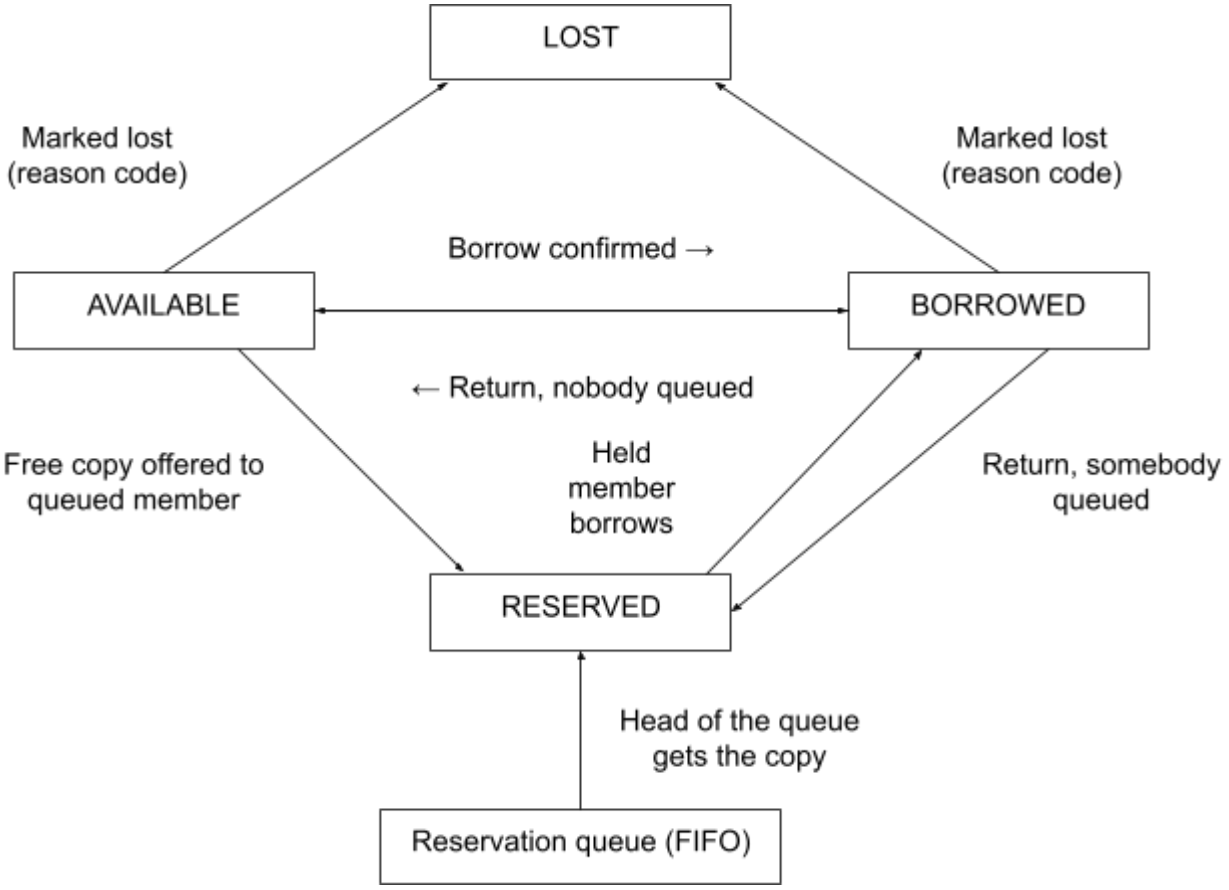
Registering a member, a book and two copies. The new member receives a key pair; the public key is the member's identity on the chain.

The state machine

A state machine is a fixed list of the states something can be in and the moves that are allowed between them. For a copy, the allowed moves are stored as a table in inventory.c, and any move that is not in the table is refused. The table below lists them.

From	To	When
AVAILABLE	BORROWED	A borrow is confirmed in a block
BORROWED	AVAILABLE	A return is confirmed and nobody is queued for the title
BORROWED	RESERVED	A return is confirmed and somebody is queued. The copy is held for them
RESERVED	BORROWED	The member the copy is held for borrows it
AVAILABLE	RESERVED	A queued member is offered a copy that is free
AVAILABLE or BORROWED	LOST	mark_lost by a librarian, which requires a reason code

Two rules keep the machine consistent. First, a copy changes state **only when its record is mined**, never when the request is typed. Second, a copy takes **one state-changing request at a time**: while its borrow is in the pool, nobody else can borrow or return it.



The copy state machine and the reservation queue that feeds the RESERVED state.

```
[UTXO | difficulty 2 | pool 0 | chain 2] library> mark_lost BK001-C1 NOT_RETURNED
LOST request REQ-000002 signed by LIB001 (nonce 0) entered the pending pool.

[UTXO | difficulty 2 | pool 1 | chain 2] library> mine_solo

=== SOLO MINING by LIB001 (difficulty 2: hash must start with 2 zeros) ===

Block #2 candidate: 1 request taken from the top of the pool: REQ-000002
Mined block #2 after 109 hash attempts (nonce 108, 0.1 ms)

--------------------------------------------------------------------------
Block #2  [LENDING]  1 record, sealed by LIB001
  Time        : 2026-10-13 16:40
  PoW         : difficulty 2, nonce 108
  Hash        : 002cbbd9d87d0209450e08bf089adc72ad36bbe1fe55a930ef7cd05b1a14e0b6
  Prev        : 008ab069bdb0fec82d1f108056fae260a77b1a68077bc78ee5eef186168f4861
  Merkle root : 8fb211b32b53a6bab75596a052c11b60d32db5f2011780ef5c9d4eb0d09e4825
  Reward      : 0 coins for returns
    REQ-000002 LOST     BK001-C1  'The Money Trap: Lost Illusions Inside th'  ALU001 (Irakoze Jean)
               at 2026-10-13 16:40 | reason NOT_RETURNED
               nonce 0 | signed by LIB001 (key 04537480a824d7c2...) | signature VALID
Confirmed 1 record:
  -> copy BK001-C1 is LOST (reason NOT_RETURNED).
1 confirmed request removed from the pending pool (0 left).

UTXO set (1 unspent output):
TX ID                 Out  Owner            Amount
-------------------- ----  ------------ ----------
867eb8c7792eb367...     0  LIB001            50.00
Total supply                                 50.00

--- Solo mining summary ---
Blocks confirmed      : 1
Total hash attempts   : 109 (average 109 per block)
Mining reward to LIB001: 50.00 coins (1 x 50), on top of the member token rewards

[UTXO | difficulty 2 | pool 0 | chain 3] library> view_copy BK001-C1

Copy BK001-C1 of BK001 'The Money Trap: Lost Illusions Inside the Tech Bubble'
  Condition      : LOST
  State          : LOST
  Current holder : (none)
  Due date       : -
  Renewals       : 0 of 2
```

The librarian override. mark_lost needs a reason code, is signed by the librarian, and takes effect only when it is mined: the copy BK001-C1 is then LOST and has no holder.

7. Reservation queue and the fines lifecycle

Reservations

When every copy of a title is on loan, a member can ask to be next in line. Each title has its own waiting line, called a reservation queue. It works first in, first out, which means that the member who reserved first is served first. In the code the queue is a linked list, where each entry stores a member_id and a reservation_date. A member joins it with the reserve_book command, and the program only allows this when no copy of the title is free.

The queue is used automatically. When a return is confirmed in a block and somebody is waiting, the copy does not go back to AVAILABLE. Instead it becomes RESERVED for the member at the front of the queue, and only that member can borrow it.

```
[UTXO | difficulty 2 | pool 0 | chain 2] library> borrow_book BK003 ALU003
No copy of 'Sustainable Leadership' is available (1 of 1 on loan, 0 on hold, 0 waiting in the queue).
Use: reserve_book BK003 ALU003

[UTXO | difficulty 2 | pool 0 | chain 2] library> reserve_book BK003 ALU003
RESERVED request REQ-000005 signed by ALU003 (nonce 0) entered the pending pool.
  Uwase Diane joins the reservation queue for 'Sustainable Leadership' once mined (0 already waiting).

[UTXO | difficulty 2 | pool 1 | chain 2] library> renew_book BK002-C1
RENEWED request REQ-000006 signed by ALU004 (nonce 1) entered the pending pool.
  Renewal 1 of 2: new due date 2026-10-28 16:29 once mined.

[UTXO | difficulty 2 | pool 2 | chain 2] library>
```

```
[UTXO | difficulty 2 | pool 0 | chain 3] library> reservation_status BK003

BK003 'Sustainable Leadership': 0 of 1 copies available
  Reservation queue (1 waiting, first in first out):
    1. ALU003 (Uwase Diane), reserved 2026-10-07 16:29

[UTXO | difficulty 2 | pool 0 | chain 3] library> view_copy BK002-C1

Copy BK002-C1 of BK002 'Wars Guns & Votes: Democracy in Dangerous Places'
  Condition      : GOOD
  State          : BORROWED
  Current holder : ALU004
  Due date       : 2026-10-28 16:29
  Renewals       : 1 of 2

[UTXO | difficulty 2 | pool 0 | chain 3] library>
```

BK003 has one copy and it is on loan, so ALU003 is told to reserve it. After mining, ALU003 is number 1 in the queue. The same screenshot shows a renewal: 7 days added, renewal 1 of 2.

```
[UTXO | difficulty 2 | pool 0 | chain 3] library> return_book BK001-C1
RETURNED request REQ-000007 signed by ALU001 (nonce 1) entered the pending pool.
  Reward transaction created: 10 coins to ALU001 (TX 4eae944244c5b109...), paid when mined.
  Return is ON TIME: no fine, reward 10 coins.

[UTXO | difficulty 2 | pool 1 | chain 3] library> return_book BK003-C1
RETURNED request REQ-000008 signed by ALU002 (nonce 1) entered the pending pool.
  Reward transaction created: 10 coins to ALU002 (TX f61a41e8980eca5a...), paid when mined.
  Return is ON TIME: no fine, reward 10 coins.

[UTXO | difficulty 2 | pool 2 | chain 3] library>
```

```
Block #3 candidate: 2 requests taken from the top of the pool: REQ-000007 REQ-000008
Mined block #3 after 54 hash attempts (nonce 53, 0.6 ms)

------------------------------------------------------------------------
Block #3  [LENDING]  2 records, sealed by LIB001
  Time        : 2026-10-07 16:29
  PoW         : difficulty 2, nonce 53
  Hash        : 0063990b4c6fad7fafc9c3bb8e919ed7a617f69563c0b2832be291389b3e4bc5
  Prev        : 006f84d4504b9a132a7a1adc12d3e0cc4f1f84f58eb0b99e0a285a9821a5b4a0
  Merkle root : 44078282ab7d3656a46d64ec2e18983dbe50899e9ff7e7ae28d11b819e80c924
  Reward      : 20 coins for returns, TX 2837572f027566ff...
    REQ-000007 RETURNED BK001-C1  'The Money Trap: Lost Illusions Inside th'  ALU001 (Irakoze Jean)
               at 2026-10-07 16:29 | due 2026-10-21 16:29 | condition NEW | reward 10 coins
               nonce 1 | signed by ALU001 (key 04e3a0d852e0a7f9...) | signature VALID
    REQ-000008 RETURNED BK003-C1  'Sustainable Leadership'  ALU002 (Joseph Habimana)
               at 2026-10-07 16:29 | due 2026-10-21 16:29 | condition NEW | reward 10 coins
               nonce 1 | signed by ALU002 (key 0442b9656c8eaf49...) | signature VALID
Confirmed 2 records:
  -> reward TX confirmed: 10 coins - 1.00 fee credited to ALU001.
  -> copy BK001-C1 is AVAILABLE again.
  -> reward TX confirmed: 10 coins - 1.00 fee credited to ALU002.
  -> copy BK003-C1 is RESERVED: offered to ALU003, next in the reservation queue.
2 confirmed requests removed from the pending pool (0 left).

UTXO set (6 unspent outputs):
TX ID                Out  Owner           Amount
-------------------- ---- ------------ ----------
867eb8c7792eb367...     0  LIB001           50.00
5c546246bcad4c0a...     0  LIB001           50.00
4eae944244c5b109...     0  ALU001            9.00
4eae944244c5b109...     1  LIBRARY           1.00
f61a41e8980eca5a...     0  ALU002            9.00
f61a41e8980eca5a...     1  LIBRARY           1.00
Total supply                              120.00

--- Solo mining summary ---
Blocks confirmed      : 1
Total hash attempts   : 54 (average 54 per block)
Mining reward to LIB001: 50.00 coins (1 x 50), on top of the member token rewards

[UTXO | difficulty 2 | pool 0 | chain 4] library> reservation_status BK003

BK003 'Sustainable Leadership': 0 of 1 copies available
  Copy BK003-C1 is on hold for ALU003 (Uwase Diane): only they can borrow it.
  Reservation queue: empty

[UTXO | difficulty 2 | pool 0 | chain 4] library>
```

The automatic offer. When ALU002's return of BK003-C1 is mined, the copy becomes RESERVED for ALU003 and the queue empties.

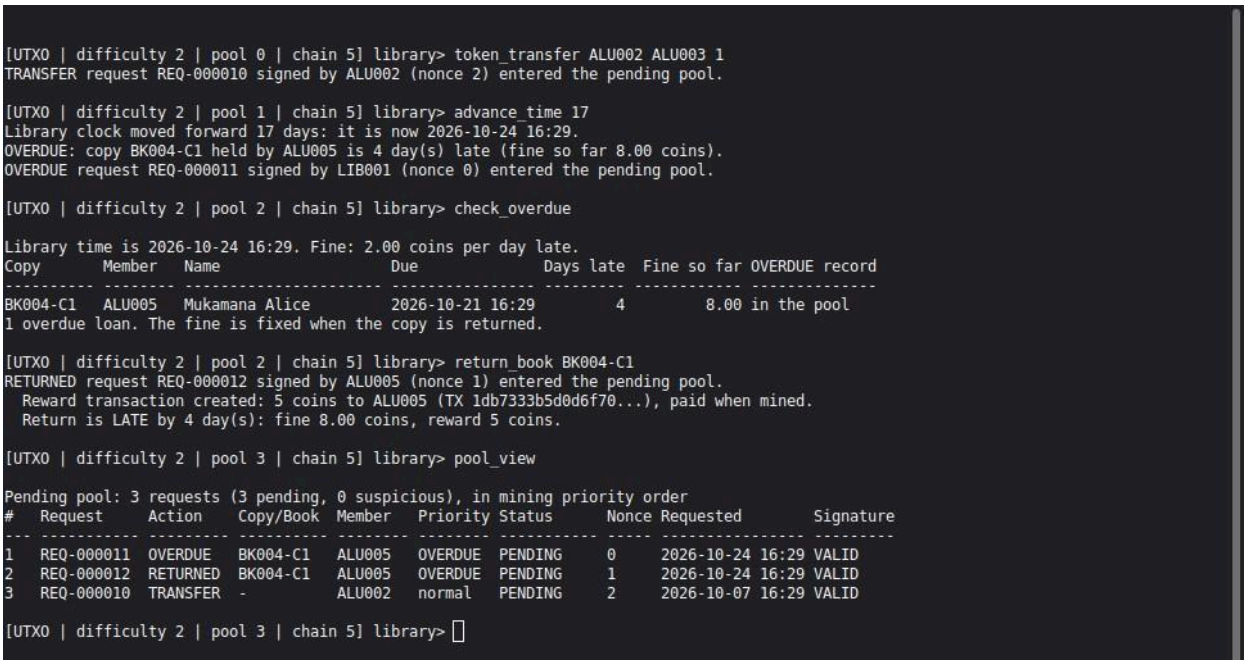
Due dates, renewals, overdue and fines

Rule	Value (config
Loan period	14 days, or a due= date at most 30 days away
Renewal	Adds 7 days, at most 2 times per loan
Renewal refused when	The loan is overdue, or another member is queued for the title
Fine	2 coins for each started day late

A late loan passes through four stages. The first is detection. Before every command, the program scans all BORROWED copies, and for each loan that has passed its due date it raises one signed OVERDUE record. This record is only a flag and creates no token transaction. The second stage is accrual: while the copy is still out, the check_overdue command shows how many days late it is and the fine so far. The third stage fixes the amount. When the copy is returned, the fine is written into the RETURNED record and is covered by the signature. Validation recomputes the fine from the due date and the return time, so it cannot be edited afterwards. The last stage is settlement. The settle_fine command creates a SETTLED record that moves tokens from the member to the

LIBRARY account, and a partial payment is allowed.

To test this without waiting two weeks, an administrator can run advance_time followed by a number of days. It moves the library's clock forward only, so block timestamps keep increasing.



After the clock moves 17 days, the loan of BK004-C1 is 4 days late: fine 8 coins, reward only 5. The pool at the bottom shows prioritisation: the transfer was requested first but the overdue-related requests are served first.

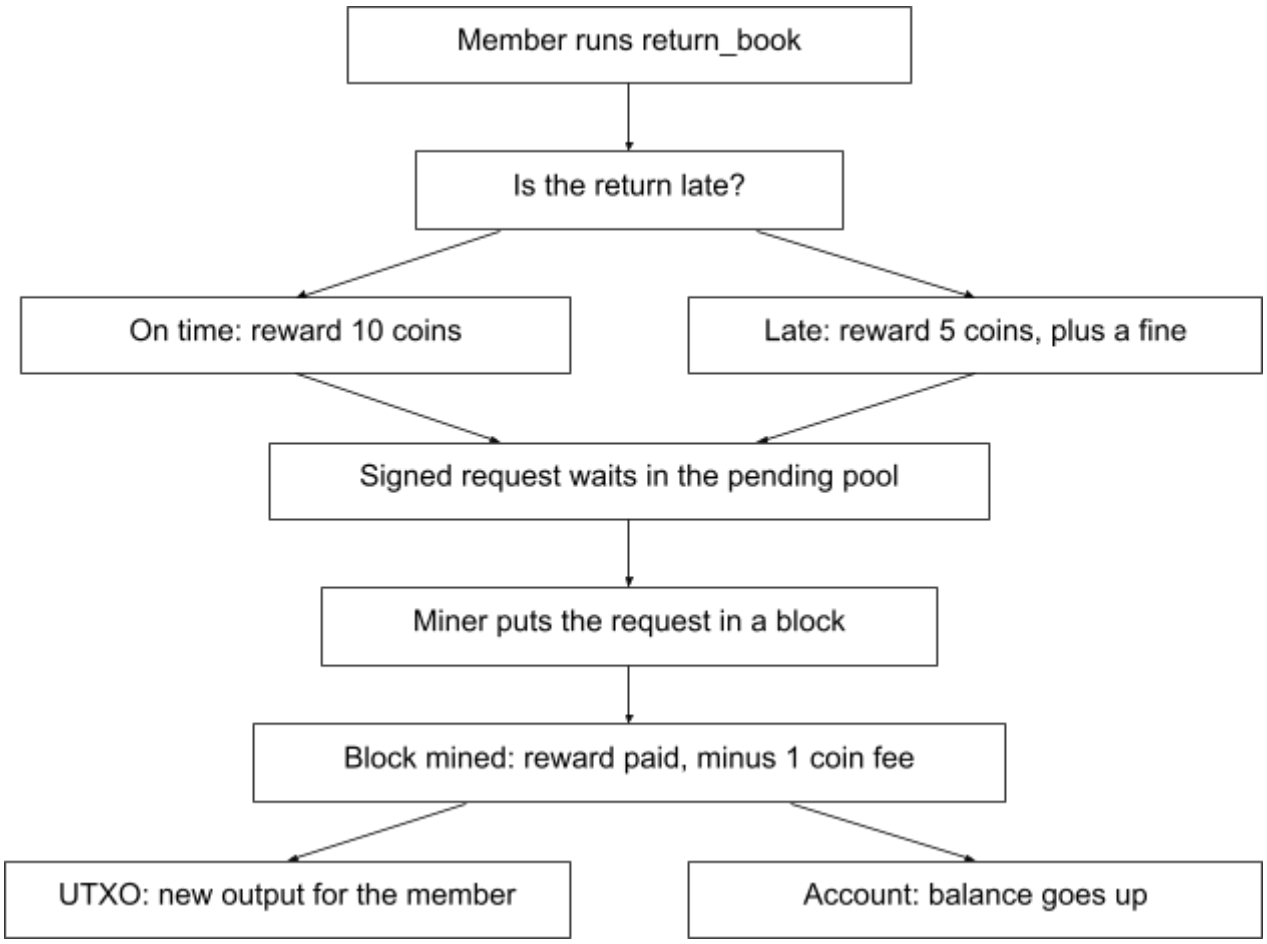
Every started day counts. The loan was due after 14 days, so 17 days later it is 3 days and a few seconds late. That counts as 4 days, and the fine is 4 x 2 = 8 coins.

8. From a return to a token reward

Tokens are the library's way of rewarding good behaviour, and returning a book is the only action that creates them. The reward is worked out when the return is requested, but it is paid only when the return is confirmed in a block.

The process starts when a member runs return_book. The program looks on the chain for the open BORROWED record of that copy and prints an error if there is none. Otherwise it compares the current time with the due_date. An on-time return carries a reward of 10 coins and a late return carries 5. The record stores the reward together with the ID of the reward transaction, which is a SHA-256 hash, and both are covered by the member's signature so that nobody can raise the reward later. The signed request then waits in the pending pool, and at this point the member's balance has not changed. When a miner includes the request in a block, the block's token_reward field is set to the total of the rewards for its returns, and its transaction_id is the SHA-256 hash of their reward transaction IDs. Only once the block is mined is the reward credited to the member, minus a fixed fee of 1 coin, in whichever ledger model is active.

A loan that is overdue and not returned produces an OVERDUE record but **no transaction**.



Token reward flow from the return request, through the pool and mining, to the balance update in either model.

Pending pool management

The pending pool is the waiting room for requests that have not been mined yet, and four operations manage it. A request is added only after it has been validated, signed and screened for misuse. The pool_view command lists every PENDING and SUSPICIOUS request with its status. After a block is mined, the requests it contains are marked CONFIRMED and removed from the pool. Finally, the pool is kept in priority order with a stable insertion sort. Overdue-related requests, meaning OVERDUE flags, returns of overdue copies and fine settlements, come first. Within the same priority the oldest request comes first, so the member who has waited longest is served first. The miner always takes the top four. The late-return screenshot in the previous section shows this ordering in a real pool.

9. The UTXO model

UTXO stands for Unspent Transaction Output. In this model tokens exist as separate “notes”, much like cash in a wallet, and a member’s balance is the sum of the notes they own.

A transfer works the way paying with cash does. It spends whole notes, called inputs, and creates new ones, called outputs. To choose the inputs, the program takes the sender’s oldest outputs until they cover the amount plus the fee. Every reward and every transfer pays a fixed fee of 1 coin, which becomes an output owned by the LIBRARY account. If the inputs are worth more than the amount plus the fee, the excess goes back to the sender as a change output, just as a shop gives change. If the sender’s outputs cannot cover the amount plus the fee, the transfer is rejected.

This model prevents double spending, which means spending the same coins twice. A spent output is removed from the set, so it cannot be spent again. Tokens already promised to requests that are still waiting in the pool are counted as well, so a member cannot promise the same coins twice before mining. The full UTXO set, with every unspent output and its owner, is printed after each confirmed block.

```
[UTXO | difficulty 2 | pool 1 | chain 4] library> mine_solo

=== SOLO MINING by LIB001 (difficulty 2: hash must start with 2 zeros) ===

Block #4 candidate: 1 request taken from the top of the pool: REQ-000009
Mined block #4 after 65 hash attempts (nonce 64, 0.4 ms)

------------------------------------------------------------------------
Block #4  [LENDING]  1 record, sealed by LIB001
  Time        : 2026-10-07 16:29
  PoW         : difficulty 2, nonce 64
  Hash        : 00260ba4ce7babb56e2e5093d39e6ae343f1732c4c3d080741078a8e7adf6418
  Prev        : 0063990b4c6fad7fafc9c3bb8e919ed7a617f69563c0b2832be291389b3e4bc5
  Merkle root : eb5878539d1549a95d30957b8e4da6f73c0f8746122320ed978ccf58ca09c337
  Reward      : 0 coins for returns
    REQ-000009 TRANSFER ALU001 -> ALU006  amount 2.00
               at 2026-10-07 16:29
               nonce 2 | signed by ALU001 (key 04e3a0d852e0a7f9...) | signature VALID
Confirmed 1 record:
  TX 4bc4acf09e0214a3...  ALU001 -> ALU006  amount 2.00  fee 1.00  nonce 2
    in : 4eae944244c5b109...:0
    out: ALU006       2.00
    out: ALU001       6.00  (change)
    out: LIBRARY      1.00  (fee)
1 confirmed request removed from the pending pool (0 left).

UTXO set (9 unspent outputs):
TX ID                 Out  Owner           Amount
-------------------- ----  ------------ ----------
867eb8c7792eb367...     0  LIB001            50.00
5c546246bcad4c0a...     0  LIB001            50.00
f6fe8ffb65e45f0d...     0  LIB001            50.00
4eae944244c5b109...     1  LIBRARY            1.00
f61a41e8980eca5a...     0  ALU002             9.00
f61a41e8980eca5a...     1  LIBRARY            1.00
4bc4acf09e0214a3...     0  ALU006             2.00
4bc4acf09e0214a3...     1  ALU001             6.00
4bc4acf09e0214a3...     2  LIBRARY            1.00
Total supply                              170.00

--- Solo mining summary ---
Blocks confirmed      : 1
Total hash attempts   : 65 (average 65 per block)
Mining reward to LIB001: 50.00 coins (1 x 50), on top of the member token rewards

[UTXO | difficulty 2 | pool 0 | chain 5] library>
```

A real UTXO transfer, mined in its own block. ALU001's single 9-coin output is spent and replaced by three: 2 to ALU006, 6 back to ALU001 as change, and the 1-coin fee. 9 = 2 + 6 + 1.

10. The account-based model

In the account model each member has one account with a balance and a nonce, much like a bank account. There are no separate notes, only one running total.

A confirmed block updates the balances directly. A reward is added to the member's balance. A transfer subtracts the amount plus the fee from the sender and adds the amount to the recipient, and it is rejected if the sender's balance is lower than the amount plus the fee. Replay protection comes from the nonce: each account has a nonce that goes up with every outgoing transaction, and a transfer submitted with a wrong or reused nonce is rejected. Each account also keeps its transactions in a linked list in memory, where every node records the sender, recipient, amount, fee and nonce. The command transaction_history followed by a member ID prints this list.

```
[Account-based | difficulty 3 | pool 0 | chain 8] library> account_balance

Account           Balance    Nonce
------------ ------------ --------
ALU001               6.00        3
ALU002               7.00        3
ALU003               1.00        1
ALU004               0.00        2
ALU005               0.00        3
LIB001             225.00        1
LIB002               0.00        0
LIBRARY              9.00        0
ALU006               2.00        4
POOL-OP              1.00        0
```

The account model. A transfer with nonce 0 is rejected because the account expects 3. After mining, the history shows the 9-coin reward (sender COINBASE, meaning newly created coins) and two transfers with their nonces.

Choosing the model

The model is chosen with a command-line option (--model utxo or --model account) or, on a new chain with no option, from a start-up menu. It applies to every token transaction of the session. If a saved chain is opened with the other model, every balance is converted: a balance becomes one unspent output, or the sum of a member's outputs becomes their balance.

11. Comparison of the two models

	UTXO model	**Account-based model**
Everyday picture	Cash: separate notes in a wallet	A bank account: one running balance
What is stored	Every unspent output and its owner	One balance and one nonce per account
Balance	Sum of the outputs you own	A single number

	UTXO model	**Account-based model**
Paying	Spend whole outputs, receive change	Subtract from one balance, add to another
Stops double spending by	Removing an output once it is spent	A nonce that must go up by one each time
Transaction size	Larger: inputs, outputs, change	Smaller: sender, recipient, amount
Auditing	Every coin can be traced back to the reward that created it	Simple to read, but coins are not individually traceable
Used by	Bitcoin	Ethereum

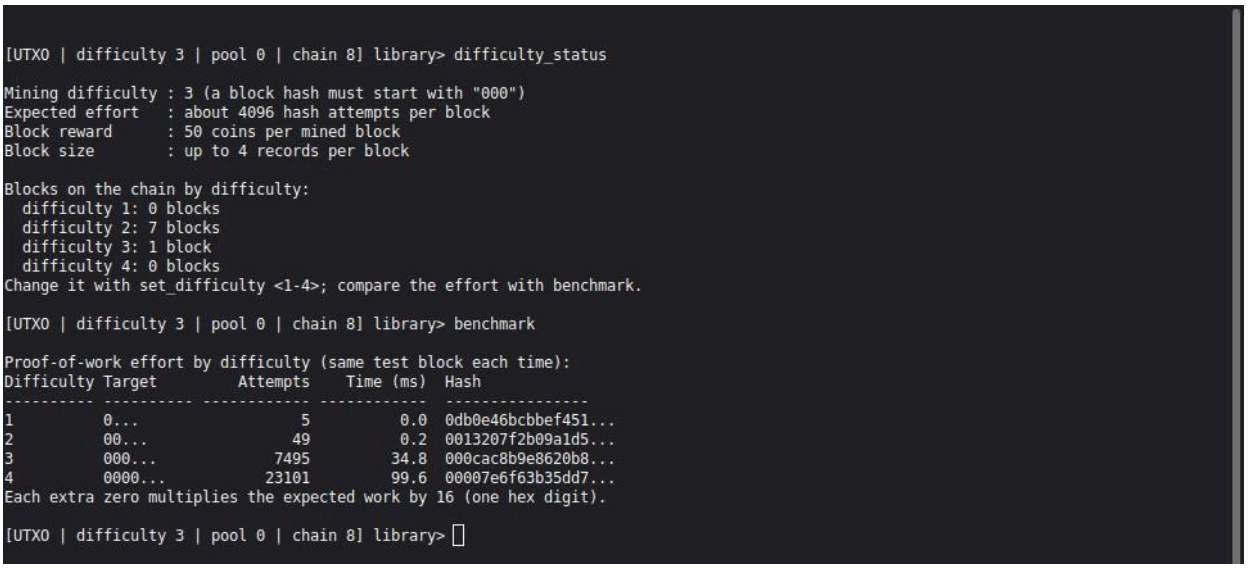
In this project, both models produce the same balances for the same chain, which I checked by switching models on a saved chain. The UTXO model is more work to implement (output selection, change, tracking outputs promised to the pool), but it makes double spending impossible by construction. The account model is simpler and gives a natural place for the nonce and the history list.

12. Mining simulation

Proof of work and difficulty

Mining turns waiting records into permanent ones by solving a puzzle that is slow to solve and instant to check. The program adds a number, the nonce, to the block header and computes the SHA-256 hash. If the hash does not start with enough zeros, it adds 1 to the nonce and tries again. The **difficulty** is the number of leading zeros required, from 1 to 4. It is set with --difficulty on the command line or with set_difficulty, and each block stores the difficulty it was mined at so that old blocks stay valid after a change.

The number of hash attempts is displayed for every block. Each extra zero multiplies the expected work by 16, because a hexadecimal character has 16 possible values.

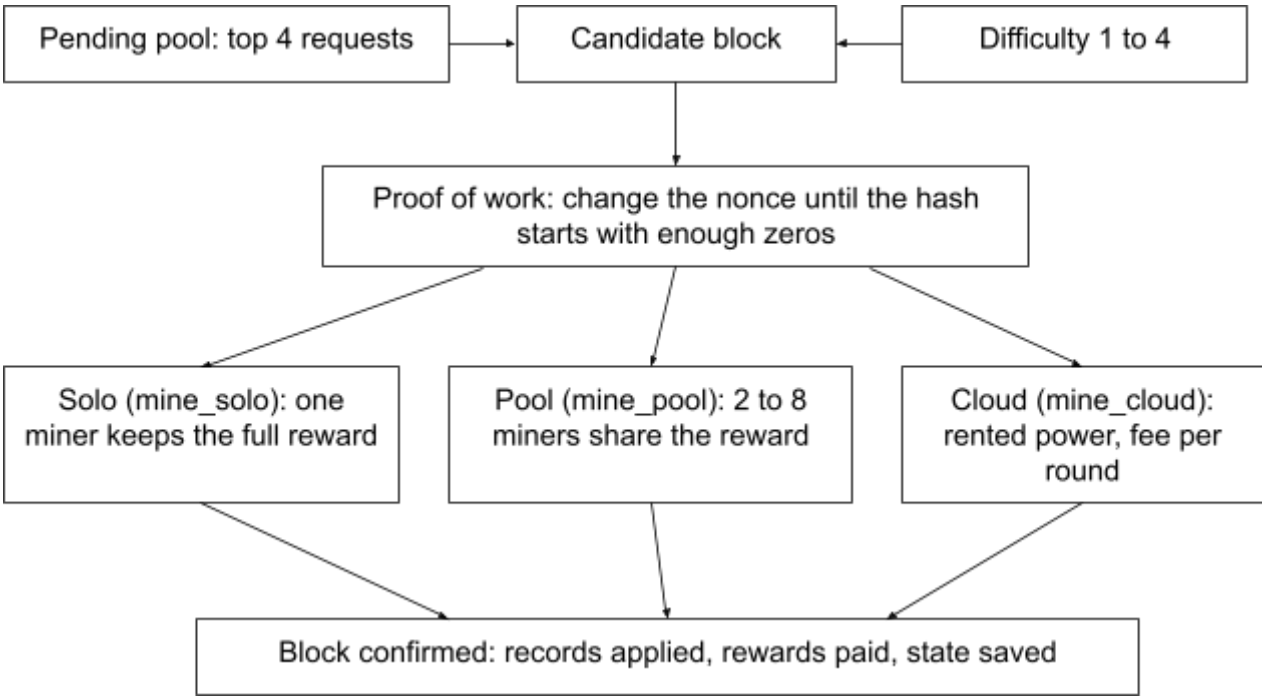


difficulty_status and the benchmark command, which mines the same test block at every

difficulty. Attempts rise from 5 at difficulty 1 to 23,101 at difficulty 4.

This cost is the point of proof of work. To change an old record, a cheat would have to redo the puzzle for that block and for every block after it, and would still fail the signature check (section 16).

All three mining methods work on the same pending pool and use the same block-building and proof-of-work code.



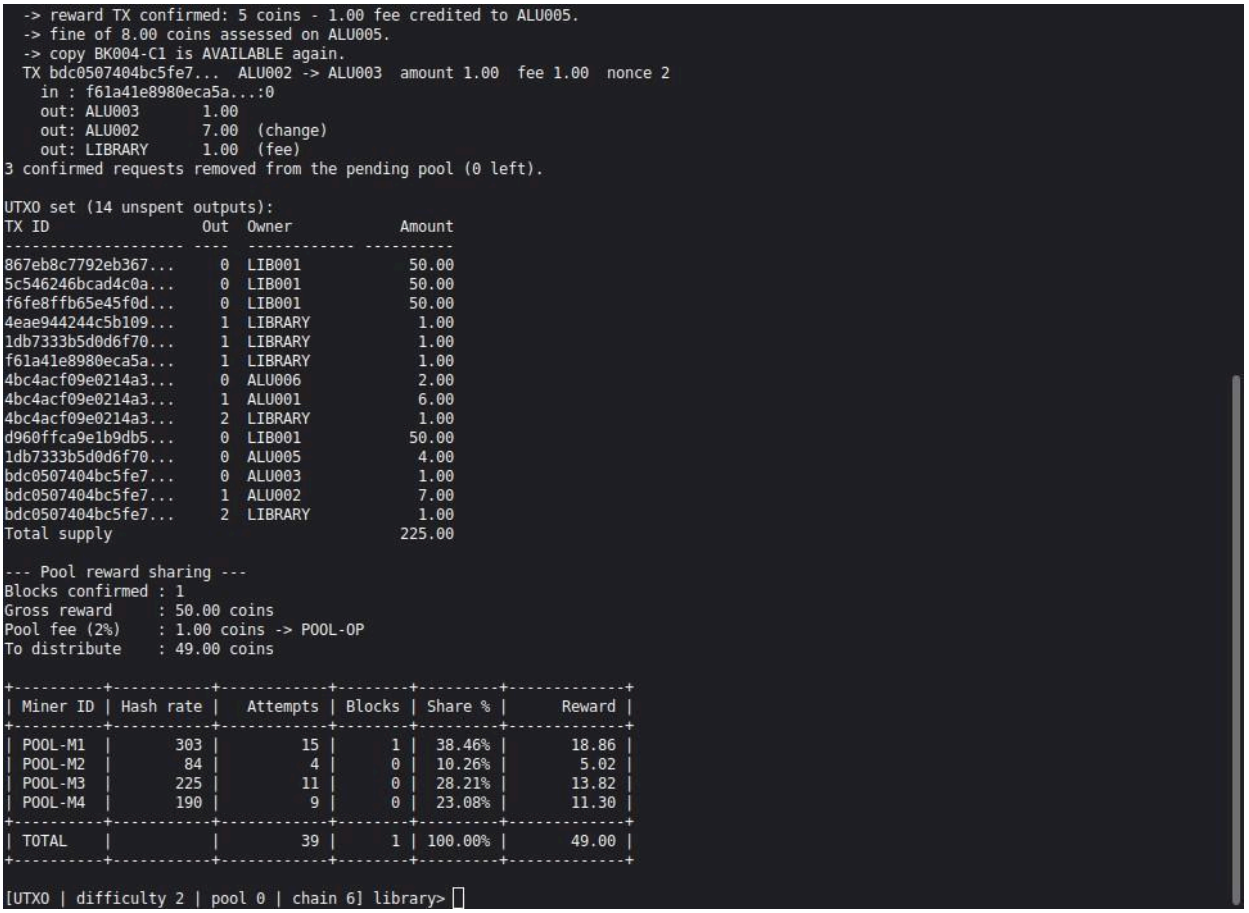
The mining workflow. One candidate block, one proof-of-work loop, and three ways to share the work and the reward.

Solo mining

With mine_solo, the logged-in librarian mines every block alone and receives the full block reward of 50 coins per block, on top of the token rewards paid to members. The four-record block shown earlier in this report was mined this way: 219 hash attempts at difficulty 2.

Pool mining

With mine_pool, between 2 and 8 simulated miners work on the same block. Each miner is given a random hash rate between 50 and 500 attempts per round. When the block is found, a pool fee of 2% is deducted from the total reward and the rest is shared according to contribution. Each miner receives (miner attempts / total attempts) x reward, so a miner who did a tenth of the work earns a tenth of the reward even if another miner found the block.



Pool mining with four miners. The 50-coin reward loses a 1-coin pool fee, and 49 coins are shared by attempts. POOL-M2 found no block, yet earns 5.02 coins for doing 10.26% of the work.

Cloud mining

With mine_cloud followed by a number of rounds (1 to 5), the user rents mining power. Each round buys 600 hashes for a fixed fee of 25 coins, and a block reward is credited whenever the rented rig confirms a block. The program prints the reward, fee and net profit of each round, then the gross earnings, total fees and net profit. It prints a warning if cumulative fees ever exceed cumulative rewards.

```
[UTXO | difficulty 2 | pool 0 | chain 6] library> view_fines ALU005

Member   Name                  Still accruing         Owed       Paid
-------- --------------------- -------------- ------------ ----------
ALU005   Mukamana Alice                 0.00         8.00       0.00
'Owed' is fixed by a late return and paid in tokens with settle_fine <member_id>.

[UTXO | difficulty 2 | pool 0 | chain 6] library> settle_fine ALU005
REJECTED: insufficient tokens: ALU005 has 4.00 available, needs 9.00 (amount + fee).

[UTXO | difficulty 2 | pool 0 | chain 6] library> settle_fine ALU005 3
SETTLED request REQ-000013 signed by ALU005 (nonce 2) entered the pending pool.

[UTXO | difficulty 2 | pool 1 | chain 6] library>
```

```
    out: LIBRARY      3.00
    out: LIBRARY      1.00  (fee)
  -> ALU005 settled 3.00 coins of fines (5.00 still owed).
1 confirmed request removed from the pending pool (0 left).

UTXO set (20 unspent outputs):
TX ID                Out  Owner            Amount
-------------------- ----  ------------ ----------
867eb8c7792eb367...     0  LIB001            50.00
5c546246bcad4c0a...     0  LIB001            50.00
f6fe8ffb65e45f0d...     0  LIB001            50.00
4eae944244c5b109...     1  LIBRARY            1.00
1db7333b5d0d6f70...     1  LIBRARY            1.00
f61a41e8980eca5a...     1  LIBRARY            1.00
4bc4acf09e0214a3...     0  ALU006             2.00
4bc4acf09e0214a3...     1  ALU001             6.00
4bc4acf09e0214a3...     2  LIBRARY            1.00
d960ffca9e1b9db5...     0  LIB001            50.00
1613a16a88f3321e...     0  POOL-M4           11.30
bdc0507404bc5fe7...     0  ALU003             1.00
bdc0507404bc5fe7...     1  ALU002             7.00
bdc0507404bc5fe7...     2  LIBRARY            1.00
9b01a7df6d8b639c...     0  POOL-OP            1.00
453e25cf5f5c81ec...     0  POOL-M1           18.86
66103235b5597d1e...     0  POOL-M2            5.02
2c3e595582abf1a1...     0  POOL-M3           13.82
63134c49107a9f16...     0  LIBRARY            3.00
63134c49107a9f16...     1  LIBRARY            1.00
Total supply                              275.00

--- Cloud mining earnings ---
+-------+--------+-----------+------------+-----------+--------------+
| Round | Blocks |    Reward | Rental fee |       Net | Cumul. net   |
+-------+--------+-----------+------------+-----------+--------------+
|     1 |      1 |     50.00 |      25.00 |     25.00 |        25.00 |
|     2 |      0 |      0.00 |      25.00 |    -25.00 |         0.00 |
|     3 |      0 |      0.00 |      25.00 |    -25.00 |       -25.00 |  <- fees exceed rewards
+-------+--------+-----------+------------+-----------+--------------+
Gross earnings  : 50.00 coins (1 block x 50)
Total fees paid : 75.00 coins (3 rounds x 25.00)
Net profit      : -25.00 coins
Rental deducted from LIB001 : 75.00 coins -> CLOUD-OP
LIB001 balance is now 175.00 coins

WARNING: this rental is UNPROFITABLE - cumulative fees exceeded cumulative rewards from round 3.

[UTXO | difficulty 2 | pool 0 | chain 7] library>
```

Fine settlement confirmed by cloud mining. Paying the full fine is rejected for lack of tokens, so the member pays 3 coins. After the payment 5 coins are still owed. The rental mines one block (50 coins) but costs 75 coins over 3 rounds, so the program warns that it is unprofitable from round 3.

13. ECDSA signature scheme

A digital signature is the electronic equivalent of a handwritten signature, except that it cannot be copied onto another document. It is made with a private key that only the signer holds, and anyone can check it with the matching public key. Formative 1 and 2 used one library key for everything. In this version every member and every librarian signs with their own key.

Key generation. When a member or librarian is first seen, the program generates an ECDSA key pair on the secp256k1 curve, which is the curve used by Bitcoin, with OpenSSL. I used OpenSSL's EVP interface because the older EC_KEY functions are deprecated in OpenSSL 3 and would produce warnings with -Wall. Two files are written to data/keys. The private key is encrypted with AES-256 under the keystore passphrase and is readable only by its owner (file mode 0600). The public key is stored in plain PEM, because it is not a secret.

Identity. The public key, as a hexadecimal string, is the member's identity on the chain. It is

stored inside every record that member signs.

Signing. Before a request enters the pool, its fields are written as one text string and signed with ECDSA over SHA-256. The private key is decrypted only for the moment it signs and is freed straight after. Members sign their own borrow, return, renew, reserve, settle and transfer requests. The logged-in librarian signs OVERDUE and LOST records.

Verification. A signature is verified three times: when the request is created, again when a miner selects it for a block, and again for every record during validate_chain. Verification uses the public key stored in the record, and also checks that this key is the one registered for that signer. So a forger cannot simply attach a key of their own.

Rejection. A record with an invalid or missing signature is rejected and never mined.

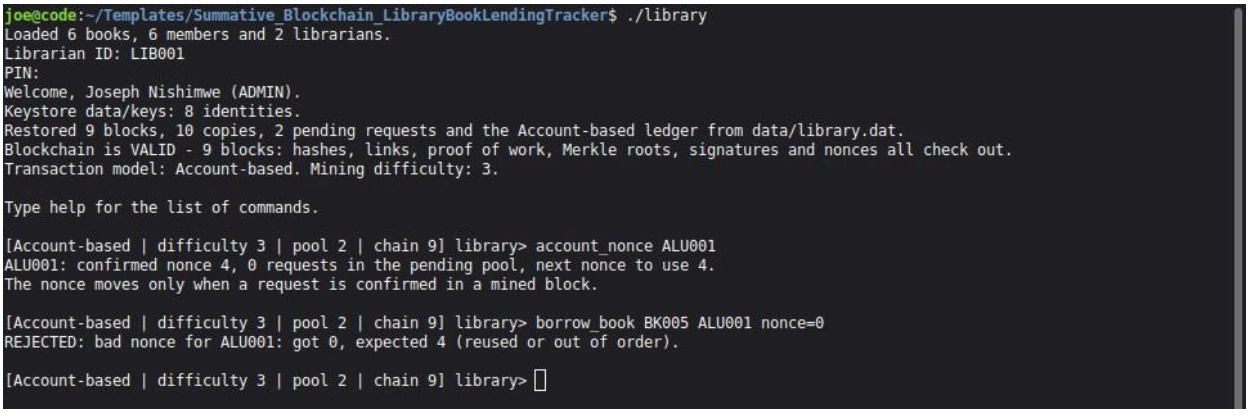
Librarian PINs are not stored either. librarians.txt holds a PBKDF2-HMAC-SHA256 hash of each PIN with 100,000 iterations.

14. Nonce-based replay protection

A signed request stays valid forever, so without protection anyone holding a copy of an old request could submit it again, for example to repeat a token transfer. This is called a replay attack, and the nonce prevents it. A nonce here is simply a counter that numbers each signer's requests.

Every member and librarian has a nonce that starts at 0. Every request must carry the signer's next nonce, which is the confirmed nonce plus the number of that signer's requests still waiting in the pool, and a request with any other nonce is rejected. The confirmed nonce advances only when a record is included in a mined block. As a final check, validate_chain requires each signer's nonces to run 0, 1, 2 and so on along the whole chain.

One nonce per identity covers both lending requests and token transfers, so member_nonce and account_nonce show the same counter. If a request is dropped from the pool, the same signer's later requests are dropped too, because their nonces can no longer become valid.

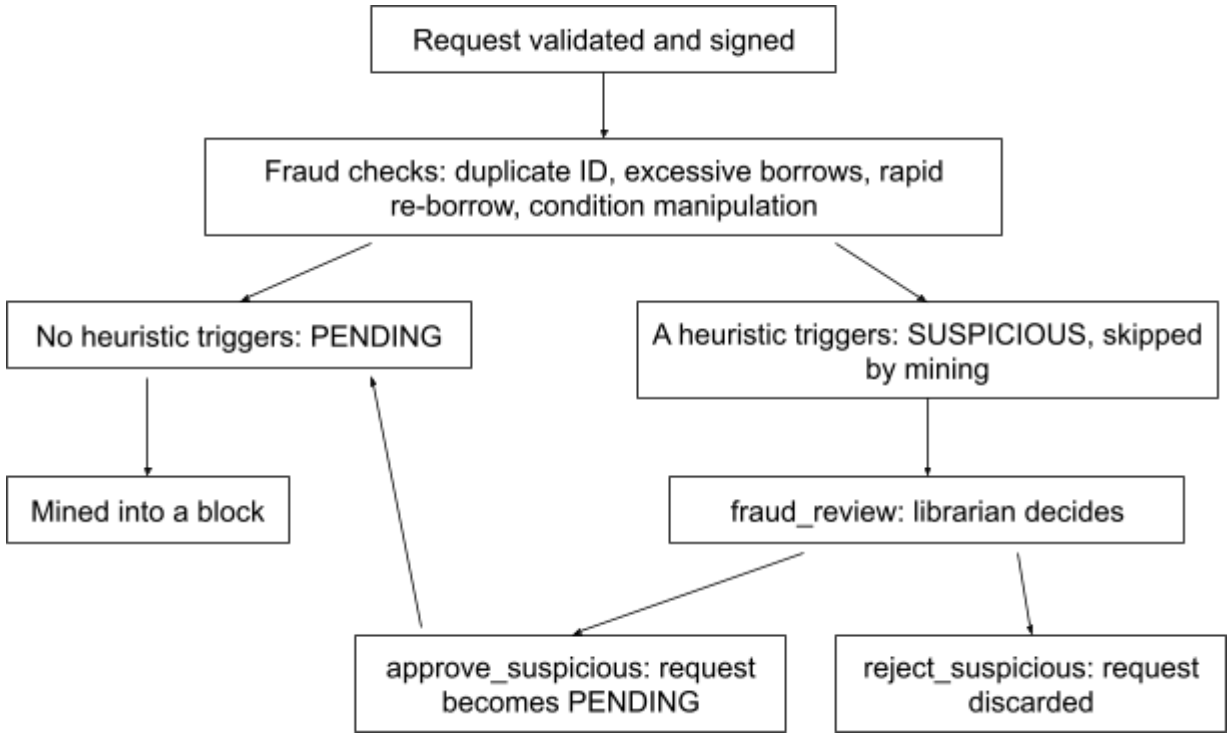


After a restart, ALU001's confirmed nonce is 4. A request replayed with nonce 0 is rejected.

15. Misuse detection and the review workflow

Some requests are valid on paper but look like misuse, for example a member borrowing far more books than usual. Every request is therefore screened by four heuristics, which are simple rules of thumb, in fraud.c before it enters the pool. The table below lists them.

Heuristic	**Conditions that trigger it**
EXCESSIVE_BORROWS	The member would hold more than 3 copies at once, counting borrows already in the pool
RAPID_REBORROW	The same member borrows the same title within 24 hours of returning it
DUPLICATE_REQUEST	The request ID already exists on the chain or in the pool
CONDITION_MANIPULATION	A return downgrades the copy's condition by more than one level with no reason code



The fraud detection pipeline, from submission to review. A flagged request is held as SUSPICIOUS until a librarian approves or rejects it.

The review workflow.

A flagged request is not rejected outright, because a heuristic can be wrong. Instead it enters the pool with the status SUSPICIOUS and the reason it was flagged. Miners skip it, so a SUSPICIOUS request is never included in a block automatically. The fraud_review command lists every held request and why it was flagged, and a librarian then decides. Running approve_suspicious with the request ID makes the request PENDING so that it can be mined, and running reject_suspicious with the ID discards it. The one exception is a duplicate request ID, which can only be rejected, because two records with the same ID can never be valid on the chain.

```
[UTXO | difficulty 2 | pool 3 | chain 7] library> borrow_book BK006 ALU006
BORROWED request REQ-000017 signed by ALU006 (nonce 3) entered the pending pool.
  FLAGGED SUSPICIOUS [EXCESSIVE_BORROWS]: ALU006 would hold 4 copies at once (maximum 3).
  It will not be mined until a librarian runs approve_suspicious REQ-000017 (or reject_suspicious REQ-000017).
  Copy BK006-C2 of 'Things Fall Apart' for Test Student, due 2026-11-07 16:29.

[UTXO | difficulty 2 | pool 4 | chain 7] library> borrow_book BK005 ALU002 id=REQ-000001
BORROWED request REQ-000001 signed by ALU002 (nonce 3) entered the pending pool.
  FLAGGED SUSPICIOUS [DUPLICATE_REQUEST]: request ID REQ-000001 already exists on the chain or in the pending pool.
  It will not be mined until a librarian runs approve_suspicious REQ-000001 (or reject_suspicious REQ-000001).
  Copy BK005-C1 of 'Gulliver's Travels and Other Writings' for Joseph Habimana, due 2026-11-07 16:29.

[UTXO | difficulty 2 | pool 5 | chain 7] library> fraud_review

Fraud review: 2 requests held as SUSPICIOUS

  REQ-000017  [EXCESSIVE_BORROWS]
    Why    : ALU006 would hold 4 copies at once (maximum 3)
    REQ-000017 BORROWED BK006-C2  'Things Fall Apart'  ALU006 (Test Student)
               at 2026-10-24 16:29 | due 2026-11-07 16:29
               nonce 3 | signed by ALU006 (key 04926b61a8a543f3...) | signature VALID
    Decide : approve_suspicious REQ-000017   or   reject_suspicious REQ-000017

  REQ-000001  [DUPLICATE_REQUEST]
    Why    : request ID REQ-000001 already exists on the chain or in the pending pool
    REQ-000001 BORROWED BK005-C1  'Gulliver's Travels and Other Writings'  ALU002 (Joseph Habimana)
               at 2026-10-24 16:29 | due 2026-11-07 16:29
               nonce 3 | signed by ALU002 (key 0442b9656c8eaf49...) | signature VALID
    Decide : approve_suspicious REQ-000001   or   reject_suspicious REQ-000001

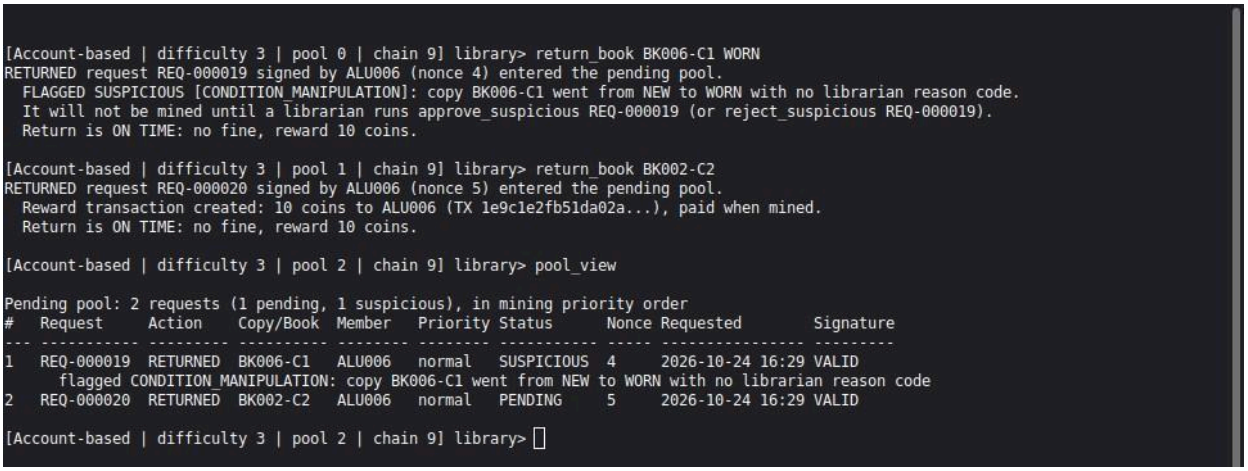
Heuristics checked on every request:
  EXCESSIVE_BORROWS       more than 3 copies held at once
  RAPID_REBORROW          same title borrowed again within 24 hours of returning it
  DUPLICATE_REQUEST       request ID already on the chain or in the pool
  CONDITION_MANIPULATION  condition down more than one level with no reason code

[UTXO | difficulty 2 | pool 5 | chain 7] library> approve_suspicious REQ-000017
Request REQ-000017 (BORROWED, flagged EXCESSIVE_BORROWS) approved by LIB001: it is PENDING and can now be mined.

[UTXO | difficulty 2 | pool 5 | chain 7] library> reject_suspicious REQ-000001
DROPPED BORROWED request REQ-000001 of ALU002: rejected by LIB001 after fraud review.

[UTXO | difficulty 2 | pool 4 | chain 7] library>
```

The fourth borrow by ALU006 is flagged for excessive borrows, and a request reusing the ID REQ-000001 is flagged as a duplicate. The librarian approves the first and rejects the second.



Condition manipulation. A copy borrowed as NEW is returned as WORN with no reason code. The request is held as SUSPICIOUS, and the pool shows the reason under it.

16. Chain validation and tamper detection

The validate_chain command answers one question: has anything been changed? It walks through the whole chain and checks every block and every record.

For every block it checks that the index is in order, starting from a genuine genesis block, and that the block hash recomputed from the header matches the stored hash. It checks that the hash satisfies the block's difficulty, which is the proof of work, and that previous_hash equals the hash of the block before it. It checks that the timestamp is strictly greater than the previous block's. Finally it checks that the Merkle root recomputed from the records matches the header and that the reward summary matches the records.

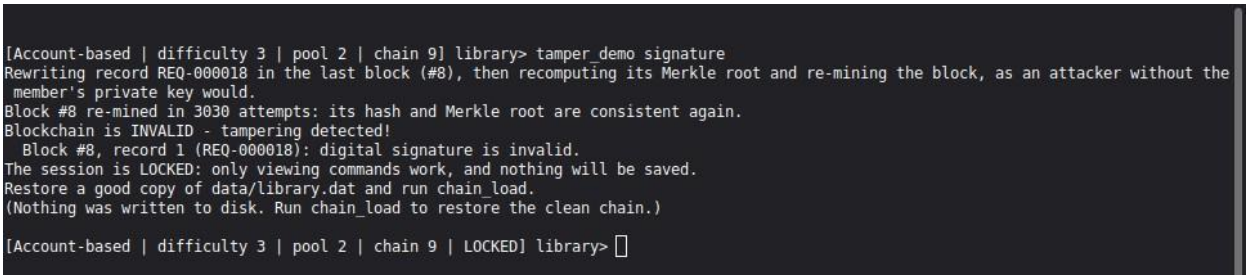
For **every record** it checks the ECDSA signature, that the signer's key is the registered one, that the book and member exist in the registries (referential integrity), that the record ID is unique, and that the nonce follows the signer's sequence.

The validation gate. A chain loaded from disk is validated before the prompt appears. If it fails, the session is LOCKED: viewing commands work, nothing else does, and nothing is saved, so a tampered state can never overwrite a good file.

The tamper_demo command (administrators only) attacks the chain in memory in three ways. The two screenshots below show the simplest attack and the strongest one.



A due date is changed inside block #1. Validation names the block and the reason, the session locks, a borrow is refused, and chain_load restores the clean chain.



A stronger attacker rewrites a record, recomputes the Merkle root and re-mines the block, so the hash and the root are consistent again. Validation still fails, because the attacker cannot produce the member's ECDSA signature.

17. Disk persistence

The whole state survives a restart. It is saved after every command that changes state, after every mined block, and on exit.

Format. I chose a plain-text structured file, data/library.dat. Each line is one record with fields separated by the | character, and the first field names the kind of line. A text format can be read and checked by eye, which a binary file cannot.

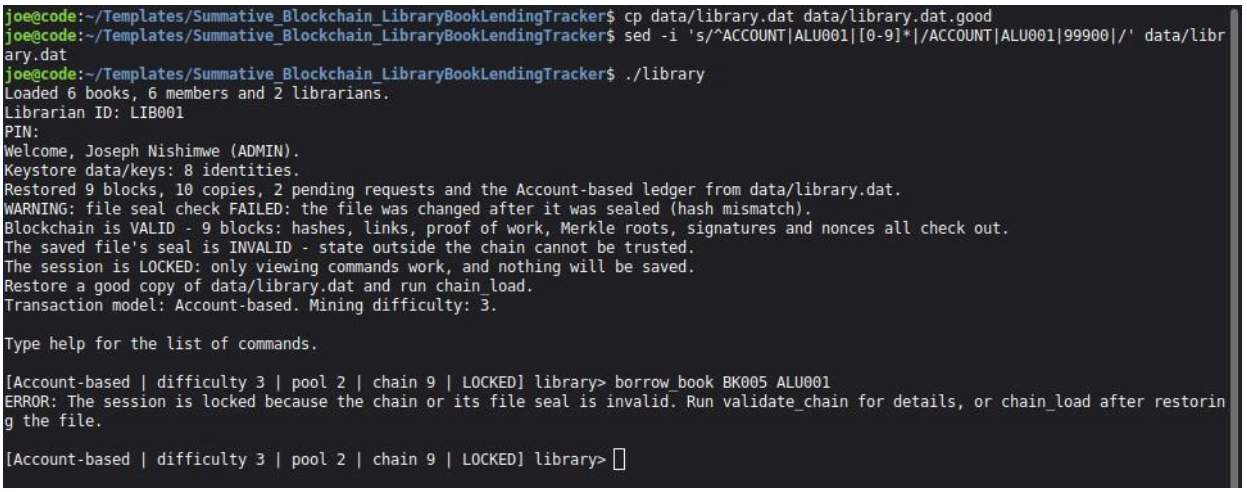
```
LIBRARY-LEDGER|version
META|difficulty|block_reward|model|clock_offset|next_request_seq|mint_sequence|chain_height
BLOCK|index|timestamp|record_count|previous_hash|merkle_root|sealed_by|token_reward|
transaction_id|difficulty|nonce|hash
RECORD|record_id|timestamp|action|book_id|copy_id|book_title|member_id|member_name|peer_id|
due_date|fine_amount|amount|condition|reason|token_reward|tx_id|nonce|signer_id|
signer_pubkey|signature_hex
COPY|copy_id|book_id|condition|state|current_holder_id|due_date|renewals|overdue_flagged
QUEUE|book_id|member_id|reservation_date
POOL|status|priority|flag|flag_detail|<record fields>
FINES|member_id|fines_owed|fines_paid
ACCOUNT|id|balance|nonce
UTXO|tx_id|output_index|owner|amount
HISTORY|account|tx_id|sender|recipient|amount|fee|nonce|has_nonce
SEAL|sha256_of_all_lines_above|librarian_id|ecdsa_signature_hex
```

What the brief requires	Line type
Full blockchain with all records	BLOCK, followed by its RECORD lines
Copy inventory and states	COPY
Reservation queues	QUEUE
Pending pool state	POOL
Token balances or UTXO set, and nonces	ACCOUNT, UTXO, HISTORY
Difficulty, block reward, chain metadata	META

Serialisation approach.
Numbers are written as decimal text, and hashes and signatures as hexadecimal. The state is first written to a temporary file, which is then renamed, so a crash cannot leave half a file behind.

The last line of the file is a seal: the SHA-256 hash of all the lines above it, signed with the librarian's ECDSA key. The chain already protects the lending records, and the seal extends that protection to the balances, the inventory and the pool, which are not on the chain.

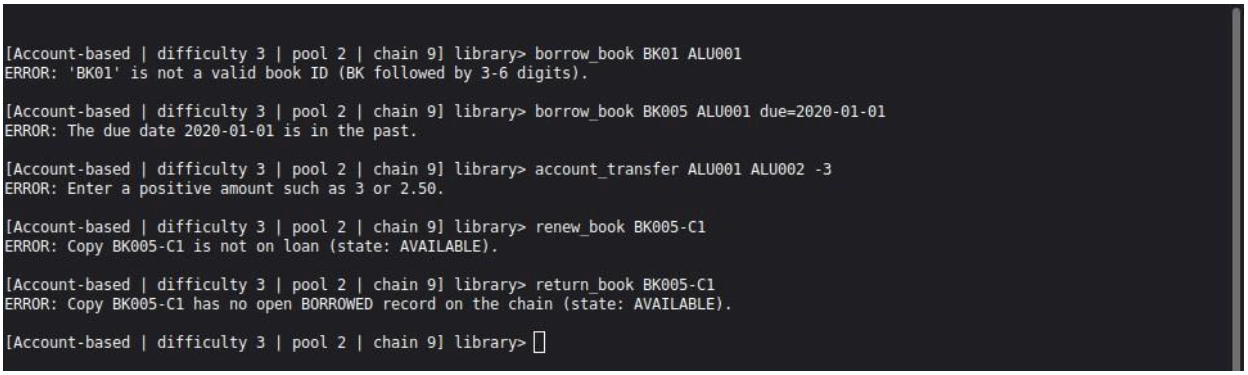
At start-up the program checks whether the file exists. If it does, the program restores the full state and re-validates the whole chain before any operation is allowed. If it does not, the program starts a new chain with a mined genesis block.



One balance in the file was edited by hand (the sed command at the top). The chain itself is still valid, but the seal check fails, so the session is locked and nothing can be done until a good file is restored.

18. Input validation and error handling

Every input is checked before anything is created, so a typing mistake produces a clear error message and never a bad record. IDs are checked for their format, which is BK or ALU followed by 3 to 6 digits, and for their existence in the registry. Dates must be well-formed, not in the past and at most 30 days away. Amounts must be positive, and they are stored as whole numbers of hundredths of a coin so that pool shares and the 2% fee are exact. The state of the copy is checked as well: a copy that is not on loan cannot be renewed or returned. Finally, the registry files must exist and must not be empty, otherwise the program prints an error and stops.



Rejected inputs: a badly formed book ID, a due date in the past, a negative amount, a renewal of a copy that is not on loan, and a return with no open BORROWED record.

19. Testing results

All tests below were run on the submitted code. Most are visible in the screenshots of this report. The program was also built with AddressSanitizer and run through the same scenarios with no memory errors and no leaks.

#	Test scenario	Command	Expected output	Actual output	Result
1	Borrow with invalid IDs	borrow_book BK999 ALU001; borrow_book BK001 ALU777	Both rejected with an error; pool unchanged	"Book ID 'BK999' is not in the book registry", "Member ID 'ALU777' is not in the member registry"; pool stays at 0	Pass
2	Four records in one block	4 x borrow_book, then mine_solo	One block with 4 records, hash starting with 00, 4 copies BORROWED	Block #1, 4 records, 219 attempts, hash 00..., 4 copies BORROWED	Pass
3	On-time return and reward	return_book BK001-C1, mine_solo, token_balance	Reward of 10 coins, 9 credited after the 1-coin fee	"10 coins - 1.00 fee credited to ALU001"; balance 9.00	Pass
4	Reservation and automatic offer	reserve_book BK003 ALU003, mine; return_book BK003-C1, mine	The returned copy becomes RESERVED for ALU003, not AVAILABLE	"copy BK003-C1 is RESERVED: offered to ALU003"; queue empty	Pass
5	Renewal	renew_book BK002-C1, mine_solo, view_copy	Due date moves 7 days later; renewals 1 of 2	Due 2026-10-21 becomes 2026-10-28; "Renewals 1 of 2"	Pass

#	Test scenario	Command	Expected output	Actual output	Result
6	Late return and fine	advance_time 17, check_overdue, return_book BK004-C1	OVERDUE record raised; 4 started days late; fine 8 coins; reward 5 coins	"4 day(s) late (fine so far 8.00 coins)"; "fine 8.00 coins, reward 5 coins"	Pass
7	Pool prioritisation	A transfer first, then the overdue requests, then pool_view	Overdue-related requests listed above the older transfer	OVERDUE and RETURNED at positions 1 and 2, TRANSFER at 3	Pass
8	Fine settlement with too few tokens	settle_fine ALU005; settle_fine ALU005 3	First rejected; partial payment accepted	"has 4.00 available, needs 9.00"; then "settled 3.00 coins of fines (5.00 still owed)"	Pass
9	Pool mining reward shares	mine_pool 4	2% fee; shares cover 100% of the work and add up to 49 coins	Fee 1.00; shares 38.46%, 10.26%, 28.21% and 23.08%; rewards total 49.00	Pass
10	Cloud mining warning	mine_cloud 3 with one request	Gross 50, fees 75, net -25, warning	Exactly these values and "UNPROFITABLE ... from round 3"	Pass
11	UTXO change and double spend	token_transfer ALU001 ALU006 2; two transfers of 5 from a 9-coin balance	Change output created; second transfer of 5 rejected	Outputs 2 + 6 change + 1 fee ; "has 3.00 available, needs 6.00"	Pass
12	Replay with a wrong nonce	account_transfer ALU001 ALU003 2 0	Rejected: bad nonce	"bad nonce for ALU001: got 0, expected 3"	Pass
13	Fraud: excessive borrows	Four borrow_book for ALU006	Fourth is SUSPICIOUS and not mined until approved	Flagged EXCESSIVE_BORROWS; mined only after approve_suspicious	Pass
14	Fraud: duplicate request ID	borrow_book BK005 ALU002 id=REQ-00000	SUSPICIOUS; rejected by the librarian	Flagged DUPLICATE_REQUEST; dropped by reject_suspicious	Pass

		1			
15	Fraud: rapid re-borrow	Return BK001, mine, borrow	SUSPICIOUS	"RAPID_REBORROW: ALU001 returned BK001 only	Pass

#	Test scenario	Command	Expected output	Actual output	Result
		BK001 again		0 minute(s) ago"	
16	Fraud: condition manipulation	return_book BK006-C1 WORN on a NEW copy	SUSPICIOUS	Flagged CONDITION_MANIPULATION	Pass
17	Tamper with a record	tamper_demo record	Chain INVALID; session locked	"Block #1: Merkle root does not match the records"; LOCKED	Pass
18	Forged record, re-mined	tamper_demo signature	Chain INVALID on the signature	"digital signature is invalid"	Pass
19	Persistence	exit, start again	Chain, copies, pool, balances and nonces restored and validated	"Restored 9 blocks, 10 copies, 2 pending requests"; chain VALID	Pass
20	File edited on disk	Change one balance in library.dat, restart	Seal check fails; session locked	"file seal check FAILED ... (hash mismatch)"; LOCKED	Pass

20. Design choices and limitations

Design choices.

Four design choices are worth explaining. The library desk holds the members' encrypted keys, like a custodial wallet, because members do not run their own software in this simulation. Token amounts are stored as whole numbers of hundredths of a coin, so there are no floating-point rounding errors. A hold does not expire: a copy held for a queued member stays RESERVED until that member borrows it. Finally, the keys, the registries and the data file belong together, so removing a member from members.txt or deleting data/keys makes the saved chain fail validation. This is by design.

Limitations.

The system also has limitations. One passphrase unlocks every key, whereas a real system would give each member their own secret. Mining is simulated on one computer: pool and cloud mining imitate several miners, but there is no network and there are no competing chains. A held request blocks its signer: while a member has a SUSPICIOUS request in the pool, their later requests cannot be mined until a librarian reviews it, because nonces must stay in order.

Mining reports this but cannot resolve it. Lastly, the block size is fixed, and a time-based window would confirm a single urgent request sooner on a busy system.

21. Conclusion

The system records lending events as signed records, screens them for misuse, orders them by priority, and confirms them in mined blocks of up to four records under a Merkle root. It tracks every physical copy through a strict state machine, serves reservations in order, charges fines for late returns, and pays token rewards in two different ledger models.

Each part has a clear role. The registries decide what is valid. ECDSA signatures prove who made each request. Nonces stop old requests from being replayed. The fraud heuristics hold doubtful requests for a human decision. Proof of work makes history expensive to rewrite. The Merkle root and the hash links make any change visible. The seal protects everything that is not on the chain. And validation checks all of it, every time the program starts.